

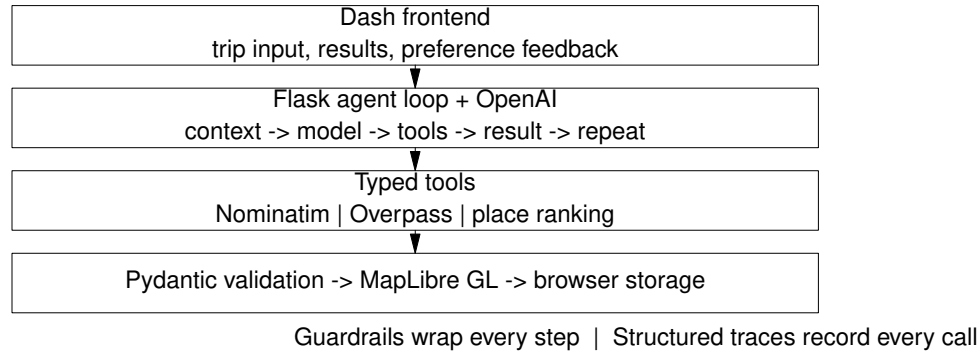
JOURNEY: ARCHITECTURE DEFENSE

Hamzah Ahmed | 24-Hour Hackathon

GOAL

Journey finds good places to visit for a destination and budget, then builds a personalized itinerary. Users refine the results through conversation until the plan matches their preferences.

ARCHITECTURE



FOUR PILLARS

1. CHAT / LOOP

Build context from trip details and current preferences. Call OpenAI, execute requested tools, validate the result, and present it. Ask what the user wants changed, update preferences, and repeat. Stop when accepted or when a turn, time, or cost limit is reached.

2. TOOLS

- **locate_destination:** resolve coordinates through Nominatim.
- **find_nearby_places:** retrieve attractions and restaurants through Overpass.
- **rank_places:** filter by budget, interests, distance, and reviews.
- **save_trip:** store the accepted plan in browser storage.

Tools return small, typed results. OpenAI decides how to combine them; tools do not contain itinerary logic.

3. GUARDRAILS

- **Input:** validate dates, travelers, budget, and text length.
- **Actions:** allow-list tools; apply timeouts, retries, rate limits, and a maximum loop count.
- **Output:** require Pydantic-validated JSON using only supplied places and valid trip dates.
- **Safety:** keep keys server-side and identify the source of place data.

4. OBSERVABILITY

Record one trace per planning session: OpenAI/tool latency, token cost, tool errors, retries, schema failures, refinement turns, and final acceptance.

TRADEOFFS

Dash provides the interactive Python UI and runs on Flask. Pydantic validates tool inputs and AI output. The design accepts browser-only storage, incomplete public place data, and variable OpenAI output. Its boundaries provide a path to richer place providers and permanent storage later.